

FastOracle: Improving Oracle Throughput and Latency via a Reduced-Size Secure Committee

AAA, BBB

Abstract—Oracle

Index Terms—Oracle, Blockchain, Committee, Throughput.

I. INTRODUCTION

ORACLE

II. RELATED WORK

III. SYSTEM DESIGN

A. System Overview

This section presents the overall architecture of the proposed FastOracle oracle protocol.

To guarantee that only highly active and reliable participants are involved in data aggregation and on-chain reporting, the protocol first computes an activity score for each miner (or validator in PoS chain) in the underlying blockchain, as formally defined in Section III-C. A dynamic threshold τ is then derived to exclude potentially inactive nodes. The set of miners whose activity scores exceed τ forms the nominating committee. This committee is then partitioned into C disjoint subcommittees with size $S_1, \dots, S_C$, where each subcommittee is responsible for handling a dedicated portion of oracle requests. (introduced in Section III-B)

The system enables concurrent request processing across all C subcommittees, substantially increasing throughput compared to sequential oracle designs. When a user submits an oracle request r by calling the corresponding on-chain smart contract, the request is appended to the tail of request queue maintained by the contract. Each subcommittee continuously monitors this queue and dequeues the next pending request r satisfying $r \bmod S_i = i$, strictly preserving FIFO order.

Upon dequeuing a request, the nodes of subcommittee independently retrieve the required off-chain data from their respective external sources. Each node locally computes its candidate value v and broadcasts the pair $\langle v, \sigma \rangle$ (where σ is its signature) to all other members of the subcommittee. Whenever a node collects valid signatures from more than half of the subcommittee members ($\geq \lceil \frac{S_i}{2} \rceil$), it then produces a compact aggregated signature σ^* via BLS multi-signature aggregation [?].

The subcommittee then employs a Verifiable Random Function [?] to generate a publicly verifiable, random permutation $\pi = (\pi_1, \pi_2, \dots)$ of its members. The first candidate π_1 is granted an initial reporting window of duration T . If a valid result $\langle v, \sigma^* \rangle$ is successfully submitted on-chain within this interval, the request is considered finalized. If no valid transaction is observed within time window T , responsibility automatically passes to the second candidate π_2 , who may

submit during the subsequent window $[T, 2T)$. This cascading fallback continues sequentially: candidate π_k is permitted to report if and only if no earlier candidate has successfully done so by time $(k-1)T$. The smart contract then verifies the correctness of the aggregated signature σ^* using the subcommittee members' public keys stored on-chain. If the signature is valid, the contract accepts this submission as the result. For all subsequent submissions, the contract directly reverts the transaction without performing further verification, minimizing gas consumption.

B. Committee Selection

In this section, we will introduce the procedure for selecting the required committee given a target total number of committee members N_{target} .

Algorithm 1 Nominating committee Construction Algorithm

Require: H ▷ Current block height.

- 1: $I_{\min} \leftarrow H - \xi + 1$
- 2: $I_{\max} \leftarrow H$ ▷ Selection window.
- 3: $d \leftarrow N_{\text{target}}$
- 4: $\mathcal{P} \leftarrow \emptyset$ ▷ Nominating committee.
- 5: **function** ISELIGIBLEMINER(p)
- 6: **if** p called becomeNode() in 10 blocks **then**
- 7: **return true**
- 8: **end if**
- 9: **return false**
- 10: **end function**
- 11: **function** GETNOMINATINGCOMMITTEE(N_{target})
- 12: **while** $d \neq 0$ **do**
- 13: **for** p who mined a block in $[I_{\min}, I_{\max}]$ **do**
- 14: **if** ISELIGIBLEMINER(p) **then**
- 15: $\mathcal{P} \leftarrow \mathcal{P} \cup \{p\}$
- 16: **end if**
- 17: **end for**
- 18: $d \leftarrow N_{\text{target}} - |\mathcal{P}|$
- 19: $I_{\max} \leftarrow I_{\min} - 1$
- 20: $I_{\min} \leftarrow I_{\max} - d$ ▷ Shift window backward.
- 21: **end while**
- 22: **return** $\mathcal{P}$
- 23: **end function**

To ensure that committee members are actively engaged participants, we introduce a voluntary selection algorithm. The nominating committee $\mathcal{P}$ is constructed iteratively according to Algorithm 1.

The algorithm begins by initializing the selection window $[I_{\min}, I_{\max}]$ to the most recent ξ blocks, we will calculate it

in Section III-D. It then iteratively processes this window to select eligible miners. In each iteration, the protocol examines all miners who minted a block within $[I_{\min}, I_{\max}]$. A miner p is added to the committee $\mathcal{P}$ if p has called the function `becomeNode()` within 10 blocks. This mechanism ensures that only actively voluntary miners are included.

After processing the current window, the algorithm calculates the remaining deficit $d = N_{\text{target}} - |\mathcal{P}|$. If $d = 0$, the size of nominating committee has reached the target and the algorithm terminates. Otherwise, the algorithm sets a new selection window of length d that immediately precedes the previous one. Specifically, if the previous window spanned blocks $[I_{\min}, I_{\max}]$, then the new window is defined as $[I_{\min} - 1 - d, I_{\min} - 1]$. This process repeats until N_{target} committee members are selected.

After constructing the nominating committee, we partition the committee into C subcommittees, with sizes $S_1, \dots, S_C$. The value of C and the specific values of S_i are calculated in Section III-E.

C. Miner Activity Score

To estimate an appropriate value for ξ , we will first measure the liveness of all miners. This section presents a methodology for computing a miner activity score based exclusively on on-chain data. The proposed score aims to provide a comprehensive characterization of miners' engagement within the blockchain ecosystem.

To achieve this, we will partition the blockchain into a sequence of non-overlapping time windows. For each time window, we compute miner's Block Production Score, Non-Contract Transaction Score, and Smart-Contract Interaction Score.

The three per-window scores are then subjected to a non-linear transformation that compresses extreme values. Finally, we apply an exponential time-decay weighting across the sequence of windows to produce the ultimate miner activity score. This formulation ensures that the final score reflects both the intensity and the recency of a miner's observable on-chain behavior.

1) *Time Window Partitioning*: To analyze miner behavior over different time period and to mitigate the impact of short-term fluctuations, we partition the entire blockchain history into consecutive, non-overlapping time windows of fixed length L blocks, measured backward from the current chain head. The most recent window (I_1) always contains the latest $\min(L, H)$ blocks, while earlier windows cover progressively older history.

Formally, let $H \in \mathbb{N}_0$ denote the height of the current chain head and $L \in \mathbb{N}^+$ the chosen window length in number of blocks. We partition the block heights $\{0, 1, \dots, H\}$ into windows I_k for $k = 1, 2, \dots, K$, where $K = \lceil (H + 1)/L \rceil$.

Definition 1 (Backward Time Windows). *The k -th window is defined as the set of block heights*

$$I_k := \{j \mid \max(H - kL + 1, 0) \leq j \leq H - (k - 1)L\} \quad (1)$$

A key advantage of this window partitioning is that active score computations across windows are completely indepen-

dent. This structural property enables highly efficient, embarrassingly parallel execution on modern GPU architectures: multiple windows can be processed concurrently with near-linear speedup, substantially reducing end-to-end computation time even for chains with very long histories.

For a miner u that mined a block at height i , the window index containing this block is $k_u = \lfloor \frac{H-i}{L} \rfloor + 1$, which means $i \in I_{k_u}$. While calculating miner u 's active score, we will only consider the k_u -th window and the subsequent windows for miner u .

2) *Three-Dimensional Activity Metrics*: To identify suitable oracle committee members, we aim to capture addresses that exhibit persistent and meaningful on-chain participation, rather than merely reflecting control over the scarce resource that determines block proposal probability (computational power in PoW or economic stake in PoS).

Toward this end, we characterize each miner's behavior during window I_k using three complementary dimensions:

- **Block Production Score** ($BS_{k,u}$): the number of blocks mined by u inside I_k .
- **Non-Contract Transaction Score** ($TS_{k,u}$): the total number of non-smart-contract transactions initiated by u within I_k .
- **Smart-Contract Interaction Score** ($SCS_{k,u}$): quantifies u 's depth of interaction with high-quality smart contracts (defined below).

Notably, while block production ($BS_{k,u}$) is an important signal of network contribution in its own right, it is predominantly a proxy for resource ownership rather than for active usage of the chain. Relying heavily on block production alone would therefore bias the committee toward large miners or stakers, at the expense of users who engage deeply with applications but rarely propose blocks.

3) *Smart-Contract Score*: To compute smart-contract interaction scores, we model user-contract interactions as a bipartite graph, use normalized gas-based weights to define transition probabilities, incorporate contract caller diversity via entropy, and iteratively update user and contract scores using a PageRank-like algorithm.

Let $\mathcal{U}$ be the set of users and $\mathcal{C}$ the set of smart contracts. We model interactions between users and contracts as a weighted bipartite graph $G = (\mathcal{U} \cup \mathcal{C}, E)$, where an edge $u \rightarrow c$ carries weight $G_{u \rightarrow c}$ equal to the total gas used by u when calling contract c (aggregated over the relevant windows).

Then we calculate the normalized weight from user u to contract c , and from contract c to user u is

$$p_{u,c} = \frac{G_{u \rightarrow c}}{\sum_{v \in \mathcal{U}} G_{v \rightarrow c}}, p_{c,u} = \frac{G_{u \rightarrow c}}{\sum_{s \in \mathcal{C}} G_{u \rightarrow s}} \quad (2)$$

, respectively.

Let $U_c \subseteq \mathcal{U}$ be the set of users that called contract c . then the normalized Shannon entropy of c 's caller distribution can be calculated as:

$$H(c) = -\frac{\sum_{u \in U_c} p_{u,c} \log p_{u,c}}{\log |U_c|} \quad (3)$$

We adopt a damped PageRank [?] iteration over the bipartite graph. Let $CAC^{(t)}(c)$ and $UAS^{(t)}(u)$ denote the

contract authority and user activity scores at iteration t , respectively. The update rules are:

$$\text{CAC}^{(t)}(c) = \frac{1-d}{|C|} + d \cdot H(c) \sum_{u \in \mathcal{U}} p_{c,u} \cdot \text{UAS}^{(t-1)}(u) \quad (4)$$

$$\text{UAS}^{(t)}(u) = \frac{1-d}{|\mathcal{U}|} + d \sum_{c \in \mathcal{C}} p_{u,c} \cdot \text{CAC}^{(t)}(c) \quad (5)$$

with uniform initialization $\text{UAS}^{(0)}(u) = \text{CAC}^{(0)}(c) = 1$ for all nodes. After sufficient iterations, we take $\text{SCS}_{k,u} = \text{UAS}^{(k)}(u)$ for miner u in window I_k .

Note that this design deliberately omits entropy-based diversity for users as typical users often interact with only a handful of popular protocols (e.g., Uniswap, Aave), so low diversity does not necessarily imply inactivity.

4) *Aggregate Activite Score*: We argue that sustained, long-term activity is more indicative of future reliability than short bursts of behavior. Therefore, we discount older windows exponentially and apply a bounded non-linear transformation to each per-window score. Define the set of metrics for miner u in the i -th window as $\mathcal{X}_{i,u} = \{\text{BS}_{i,u}, \text{SCS}_{i,u}, \text{TS}_{i,u}\}$. The final activity score for a miner u is

$$\text{AS}_u = \sum_{k=h_u}^{\lceil H/L \rceil} \sum_{x \in \mathcal{X}_{i,u}} \frac{2}{\pi} \arctan(x) \cdot \exp(-\lambda kL) \quad (6)$$

where $\lambda > 0$ is a decay hyper-parameter. The arctan transformation bounds each dimension, preventing extreme values from dominating the score.

Because the activity score incorporates exponential decay, the score assigned to a miner is increasing in the height of the blocks they mine. Formally, we establish the following lemma:

Lemma 1 (Recent Miner Score). *Let h_a and h_b denote the height of block minted by miner a and b , respectively. If*

$$h_a - h_b > (1 + \ln 6 - \frac{\ln(\exp(\lambda L) - 1)}{\lambda L})L \quad (7)$$

then $\text{AS}_a > \text{AS}_b$.

Proof. We first derive a lower bound on AS_a and an upper bound on AS_b .

Let k_a and k_b denotes the window index that contains h_a and h_b , respectively. Since scores are non-negative and miner a must mint a block, thus $\text{BS}_{k_a,a} \geq 1$, then we have

$$\begin{aligned} \text{AS}_a &\geq \frac{2}{\pi} \arctan(\text{BS}_{k_a,a}) \exp(-\lambda k_a L) \\ &\geq \frac{1}{2} \exp(-\lambda k_a L) \end{aligned} \quad (8)$$

For the upper bound, note that each per-window score satisfies

$$\sum_{x \in \mathcal{X}_{i,u}} \frac{2}{\pi} \arctan(x) < 3, \quad (9)$$

as $\frac{2}{\pi} \arctan(x) < 1$ for each of the three metrics. Thus,

$$\begin{aligned} \text{AS}_b &< \sum_{k=k_b}^{\lceil H/L \rceil} 3 \exp(-\lambda kL) \\ &= 3 \exp(-\lambda k_b L) \sum_{j=0}^{\lceil H/L \rceil - k_b} \exp(-\lambda Lj) \\ &< 3 \exp(-\lambda k_b L) \sum_{j=0}^{\infty} \exp(-\lambda Lj) \\ &= \frac{3 \exp(-\lambda k_b L)}{1 - \exp(-\lambda L)}, \end{aligned} \quad (10)$$

To ensure $\text{AS}_a > \text{AS}_b$, it suffices that

$$\frac{1}{2} \exp(-\lambda k_a L) > \frac{3 \exp(-\lambda k_b L)}{1 - \exp(-\lambda L)}, \quad (11)$$

which yields:

$$\begin{aligned} k_b - k_a &> \frac{1}{\lambda L} \ln \left(\frac{6}{1 - \exp(-\lambda L)} \right) \\ &= \frac{1}{\lambda L} (\ln 6 + \ln(\exp(\lambda L) - 1)) \\ &= \ln 6 - \frac{\ln(\exp(\lambda L) - 1)}{\lambda L} \end{aligned} \quad (12)$$

Let $\Delta = \ln 6 - \frac{\ln(\exp(\lambda L) - 1)}{\lambda L}$. Now, $k_b - k_a = \lfloor \frac{H-h_b}{L} \rfloor - \lfloor \frac{H-h_a}{L} \rfloor$. Since $\lfloor x \rfloor - \lfloor y \rfloor > x - y - 1$. A sufficient condition for this difference to exceed $\Delta_{a,b}$ is

$$k_b - k_a > \Delta + 1 \iff \frac{h_a - h_b}{L} > \Delta + 1, \quad (13)$$

for $x > y \geq 0$. Thus, $h_a - h_b > (\Delta + 1)L$ guarantees the desired inequality. $\square$

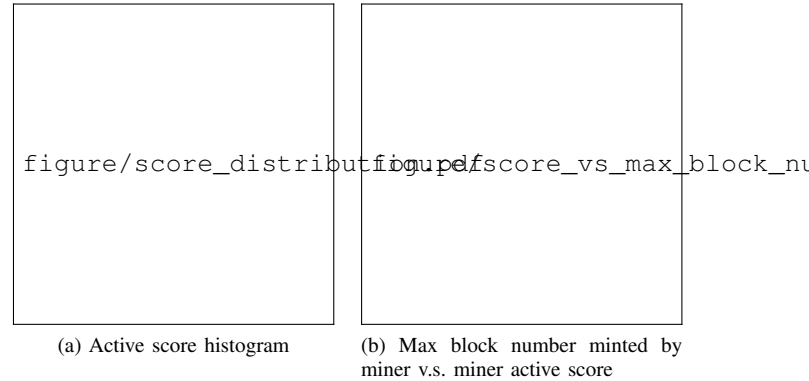


Fig. 1. Active score figure

We crawled and processed transaction data from Etherscan [?] for blocks ranging from height 24,236,736 to 24,266,735, i.e. 30,000 blocks in total. Employing an exponential decay parameter $\lambda = 0.2$ and performing 5 iterations of smart contract score calculation, with a block interval $L = 1000$, we computed the active score for each miner throughout this epoch.

Fig. 1a displays the histogram of active scores across all miners. The distribution reveals a pronounced long-tail

shape: the vast majority of miners exhibit very low active scores, while a small number of highly active miners achieve significantly larger values, resulting in a heavy right tail. This skewness is characteristic of highly unequal participation in PoS blockchain.

Fig. 1b depicts a scatter plot relating each miner's active score to the highest block height they have mined. A clear negative trend emerges: miners with smaller distances (i.e., those who have remained active closer to the current chain tip) tend to exhibit higher active scores. This empirical relationship also provides supporting evidence for Lemma 1.

D. Determination of Selection Window

In this section, we propose a dynamic adjustment mechanism to determine the parameter ξ . At the current block height H , ξ is defined as the shortest trailing window that contains at least N_{target} miners whose activity score exceeds a given threshold τ . Formally,

$$\xi = \min_{k \in \mathbb{N}^+} \text{s.t. } |\{p \in \mathcal{M}_{[H-k+1, H]} \mid AS_p > \tau\}| \geq N_{\text{target}}, \quad (14)$$

where $\mathcal{M}_{[a, b]}$ denotes the set of miners that minted at least one block in the range $[a, b]$, and AS_p is the activity score of miner p .

To bootstrap the mechanism, the initial threshold $\tau^{(1)}$ is set to the median activity score of all miners active in a recent initialization window of length W_{init} :

$$\tau^{(1)} = \text{median}\{AS_p \mid p \in \mathcal{M}_{[H-W_{\text{init}}+1, H]}\}. \quad (15)$$

The threshold τ is then adaptively updated at the beginning of each committee selecting epoch, with the explicit objective of driving the realized nominating committee size $|\mathcal{P}|$ toward the target value N_{target} . Let $\tau^{(e)}$ and $|\mathcal{P}^{(e)}|$ denote the threshold employed and the resulting nominating committee size observed during epoch e , respectively. The threshold for the subsequent epoch is then updated according to the simple proportional control rule:

$$\tau^{(e+1)} = \tau^{(e)} + \eta \cdot (N_{\text{target}} - |\mathcal{P}^{(e)}|) \quad (16)$$

where $\eta > 0$ is a hyper-parameter represents the learning rate.

This update strategy is intentionally designed to provide negative feedback that stabilizes committee size around the desired setpoint. Specifically, when the nominate committee size $|\mathcal{P}^{(e)}|$ exceeds N_{target} , the positive error term $(N_{\text{target}} - |\mathcal{P}^{(e)}|)$ becomes negative, which decreases τ in the next epoch. A lower τ relaxes the activity score requirement, thereby shortening the minimal window length $\xi^{(e+1)}$ needed to reach N_{target} active miners and ultimately reducing the nominating committee size. Conversely, when $|\mathcal{P}^{(e)}| < N_{\text{target}}$, the error is positive, τ is increased, the selection criterion becomes stricter, ξ lengthens, and more miners are included to push the nominating committee size upward.

By jointly adapting ξ and τ in a closed-loop manner, the proposed mechanism offers the following key advantages:

- **Elimination of manual calibration brittleness** Fixed thresholds are inherently suboptimal across diverse network conditions and require frequent off-chain governance intervention. The adaptive rule continuously refines τ based on the observed nominating committee size, achieving a more precise operating point relative to N_{target} without human tuning.
- **Robustness to temporal fluctuations in miner activity** Blockchains exhibit large variations in aggregate participation due to economic cycles. By dynamically adjusting τ in proportion to the deviation from the target pool size after each epoch, the protocol automatically aligns the eligibility criterion with the prevailing activity level.

Computing the active score for every miner in each election round would incur prohibitively high computational overhead. The following theorem allows us to drastically reduce this cost in our scheme:

Theorem 1. Let $\tau \in \mathbb{R}$ be a fixed activity threshold, and define the selected miner set as $S_\tau := \{u \mid AS_u > \tau\}$ and $|S| = N_{\text{target}}$. Then:

$$\forall w \in \text{miner}, \quad h_w \geq N_{\text{target}} + \Delta \Rightarrow w \notin S_\tau.$$

Proof. Consider the miner set $S = \{w \mid h_w \leq N_{\text{target}}\}$.

- if $\exists i \in S, i \notin S_\tau$, then $AS_i \leq \tau$. According to Lemma 1, $\forall w \in \text{miner}, h_w \geq N_{\text{target}} + \Delta \geq i + \Delta, AS_w \leq \tau$. Then $w \notin S_\tau$.
- if $S \subseteq S_\tau$, since $|S| = N_{\text{target}} = |S_\tau|$, then $S = S_\tau$, then $\forall w \in \text{miner}, h_w \geq N_{\text{target}} + \Delta > N_{\text{target}}, w \notin S = S_\tau$.

this concludes the proof. $\square$

Thus, we will only calculate miners who minted block at height ranging from $H - \Delta - N_{\text{target}}$ to H .

The initial committee is selected through a standard Proof-of-Work mechanism. For all subsequent committee selection procedure, the outgoing committee calculates the active scores of miners and determine ξ and τ . Because the on-chain data is identical and deterministic for all nodes, every committee member computes the exact same active score for each miner. Since the calculation logic are fully deterministic and publicly verifiable, the committee members will calculate exact the same ξ and τ .

E. Partitioning Scheme

In this section, we will calculate the total number of subcommittee (C), and each subcommittee size ($S_1, \dots, S_C$).

1) *Problem Formulation.* We model this problem as a mathematical optimization problem. Let p denote the probability that any member is adversarial. A subcommittee is considered compromised if the adversarial node controls more than half of its members. Letting S_j denote the number of nodes in the j -th subcommittee, the probability $\text{PF}(S_j)$ that the committee j is compromised is:

$$\text{PF}(S_j) = \sum_{i=\lceil S_j/2 \rceil}^{S_j} \binom{S_j}{i} p^i (1-p)^{S_j-i} \quad (17)$$

Let P_{all} denote the probability that `FastOracle` is compromised. In our protocol, security requires that all committees behave honestly. Since selecting nominating committee fails with probability at P_{select} due to chain quality. Let C denotes the number of subcommittees. The overall compromise probability satisfies:

$$P_{\text{all}} = 1 - (1 - P_{\text{select}}) \prod_{j=1}^C (1 - \text{PF}(S_j)) \quad (18)$$

For any committee partitioning scheme $\Pi = [S_1, \dots, S_C]$, S_j and C must satisfy the following requirements:

$$\forall i \in [1, C], 1 \leq S_j \leq N_{\text{target}}, \sum_{j=1}^C S_j = N_{\text{target}} \quad (19)$$

In order to improve throughput while satisfying α -bit security, We aim to find Π^*, C that is lexicographically minimal with respect to the objective vector $(-C, P_{\text{all}})$. Formally, we define the problem as:

$$\begin{aligned} &\text{lex-min} && (-C, P_{\text{all}}) \\ &\text{s.t.} && P_{\text{all}}(\Pi) \leq 2^{-\alpha}, \\ &&& \sum_{i \in \Pi} i = N_{\text{target}}. \end{aligned} \quad (20)$$

2) *Solution Approach*: We solve this problem in two steps. First, for a given number of subcommittee C , we solve out the optimal partition scheme. Second, through theoretical analysis, we derive the upper and lower bounds of the optimal C . We then perform a traversal within this bounded interval to obtain the globally optimal value of C .

Theorem 2. $1 - \text{PF}(S)$ is a log-concave function when $S \rightarrow \infty$

Proof. PF is defined as the right-tail cumulative distribution function. Bahadur–Rao theorem [?] gives that

$$\text{PF}(S) \sim \frac{c(p)}{\sqrt{S}} \exp(-S I(1/2)), \quad n \rightarrow \infty \quad (21)$$

Let $g(S) = \frac{c}{\sqrt{S}} e^{-SI(1/2)}$, let $h(N) = 1 - g(N)$. The second derivative of $\ln h$ is given by $(\ln h)'' = \frac{h''h - (h')^2}{h^2}$. Since $h' = -g'$ and $h'' = -g''$, it follows that $h''h - (h')^2 = -g''(1 - g) - (g')^2$. Given the expression $\ln g = \ln C - IN - \frac{1}{2} \ln N$, taking the second derivative on both sides yields $g \cdot g'' = \frac{g^2}{2N^2} + g'^2 > 0$. Furthermore, since $1 > g > 0$, it follows that $g'' > 0$. Consequently, $-g''(1 - g) - (g')^2 < 0$, which implies $(\ln h)'' < 0$, which means $1 - \text{PF}(S)$ is a log-concave function when $S \rightarrow \infty$. $\square$

Theorem 3 (Optimal partition scheme). *If $1 - \text{PF}(S_j)$ is a log-concave function, for certain C , the optimal partition method is*

$$\Pi = \underbrace{\lfloor \frac{N_{\text{target}}}{C} \rfloor}_{y \text{ times}}, \underbrace{\lfloor \frac{N_{\text{target}}}{C} \rfloor + 1}_{N_{\text{target}} - y \text{ times}} \quad (22)$$

, where $y = C \lfloor \frac{N_{\text{target}}}{C} \rfloor + C - N_{\text{target}}$.

Proof. Since the number of partition schemes is finite, there must exist an optimal scheme Π^* . $\forall a, b \in \Pi^*$, if $|a - b| \geq 2$, Assume without loss of generality that $a \geq b + 2$, then $(1 - \text{PF}(a))(1 - \text{PF}(b)) \geq (1 - \text{PF}(a - 1))(1 - \text{PF}(b + 1))$. This contradicts the assumption that Π^* is optimal.

Let x be the minimum value of Π^* , then value in Π must be x or $x + 1$. Assume the number of x in Π^* is y . Then we have: $xy + (x + 1)(C - y) = N_{\text{target}}$, which means $Cx + C - y = N_{\text{target}}$.

Since $1 \leq y \leq C$, $\frac{N_{\text{target}}}{C} - 1 < x \leq \frac{N_{\text{target}}}{C}$, since x is an integer, we have $\lfloor \frac{N_{\text{target}}}{C} \rfloor - 1 < x \leq \lfloor \frac{N_{\text{target}}}{C} \rfloor$, then $x = \lfloor \frac{N_{\text{target}}}{C} \rfloor$, $y = C \lfloor \frac{N_{\text{target}}}{C} \rfloor + C - N_{\text{target}}$. $\square$

To determine the optimal value of C , the subsequent theorems establish upper and lower bounds on C . The optimal value of C is then identified by searching within this feasible interval. In addition, to simplify the complex expressions, we introduce the following symbol table to encapsulate recurring expressions:

symbol	expression
L	$\ln(1 - 2^{-\alpha}) - \ln(1 - P_{\text{select}})$
β	$\ln(2p)$
D	$D_{\text{KL}}(\frac{1}{2} \ p) \cdot N_{\text{target}}$

TABLE I
NOTATION SUMMARY

Lemma 2 (Lower bound of P_{all}).

$$P_{\text{all}} \geq 1 - (1 - P_{\text{select}}) \exp\left(-\frac{C}{2}(2p)^{N_{\text{target}}/C}\right)$$

Proof. Since $p < 1/2$, it holds that $(1 - p)^x > p^x$ for all $x > 1$. Then:

$$\text{PF}_j \geq \sum_{i=\lceil S_j/2 \rceil}^{S_j} \binom{S_j}{i} p^{S_j} \geq \frac{1}{2} (2p)^{S_j} \quad (23)$$

According to **Jensen's inequality** [?], the failure probability P_{all} can then be lower-bounded as follows:

$$\begin{aligned} P_{\text{all}} &= 1 - (1 - P_{\text{select}}) \prod_{j=1}^C (1 - \text{PF}(S_j)) \\ &\geq 1 - (1 - P_{\text{select}}) \prod_{j=1}^C (1 - \frac{1}{2} (2p)^{S_j}) \\ &\geq 1 - (1 - P_{\text{select}}) \left(1 - \frac{1}{2} (2p)^{N_{\text{target}}/C}\right)^C \end{aligned} \quad (24)$$

Since $(1 - a)^C < e^{-aC}$, we have:

$$P_{\text{all}} > 1 - (1 - P_{\text{select}}) \exp\left(-\frac{C}{2} (2p)^{N_{\text{target}}/C}\right) \quad (25)$$

This concludes the proof. $\square$

Theorem 4 (Upper bound of C). *To satisfy $P_{\text{all}} \leq 2^{-\alpha}$, the number of committees C must satisfy the following upper bound:*

$$C \leq \left\lfloor -\frac{\beta N_{\text{target}}}{W_0\left(\frac{\beta N_{\text{target}}}{2L}\right)} \right\rfloor \quad (26)$$

Where $W_0(x)$ is the main branch of Lambert W function [?]. □

Proof. According to Lemma 2, we have:

$$2^{-\alpha} \geq P_{\text{all}} \geq 1 - (1 - P_{\text{select}}) \exp\left(-\frac{C}{2}(2p)^{N_{\text{target}}/C}\right) \quad (27)$$

Then we have:

$$\begin{aligned} \frac{C}{2}(2p)^{N_{\text{target}}/C} &\leq -L \\ \iff C \exp\left(\frac{\beta N_{\text{target}}}{C}\right) &\leq -2L \\ \iff -\frac{\beta N_{\text{target}}}{C} \exp\left(-\frac{\beta N_{\text{target}}}{C}\right) &\geq \frac{\beta N_{\text{target}}}{2L} \end{aligned} \quad (28)$$

Since $x = -\frac{\beta N_{\text{target}}}{C} > 0$, $y = \frac{\beta N_{\text{target}}}{2L} > 0$. $xe^x \geq y \iff x \geq W_0(y)$, we have:

$$C \leq -\frac{\beta N_{\text{target}}}{W_0\left(\frac{\beta N_{\text{target}}}{2L}\right)} \quad (29)$$

□

Lemma 3 (Upper Bound of P_{all}).

$$P_{\text{all}} \leq P_{\text{select}} + (1 - P_{\text{select}})C \exp\left(-\frac{D}{C}\right)$$

Proof. In our protocol, the size of each committee S_j satisfies $S_j \geq N_{\text{target}}/C$. Applying **Chernoff's Bound** [?], we have:

$$\text{PF}(S_j) \leq \exp(-D_{\text{KL}}(\frac{1}{2}\|p) \cdot S_j) \leq \exp\left(-\frac{D}{C}\right) \quad (30)$$

Then we have:

$$P_{\text{all}} \leq 1 - (1 - P_{\text{select}}) \left(1 - \exp\left(-\frac{D}{C}\right)\right)^C$$

Applying **Bernoulli's inequality** [?] $(1 - x)^C > 1 - Cx$, we have:

$$P_{\text{all}} \leq P_{\text{select}} + (1 - P_{\text{select}})C \exp\left(-\frac{D}{C}\right)$$

□

Theorem 5 (Lower bound of C). *if*

$$C = \left\lfloor \frac{D}{W_0\left(\frac{D}{1-\exp(L)}\right)} \right\rfloor = C_{\min}$$

. Then the inequality $P_{\text{all}} \leq 2^{-\alpha}$ must holds.

Proof.

$$\begin{aligned} P_{\text{select}} + (1 - P_{\text{select}})C \exp\left(-\frac{D}{C}\right) &\leq 2^{-\alpha} \\ \iff 1 - C \exp\left(-\frac{D}{C}\right) &\geq \exp(L) \\ \iff C \exp\left(-\frac{D}{C}\right) &\leq 1 - \exp(L) \\ \iff \frac{D}{C} \exp\left(\frac{D}{C}\right) &\geq \frac{D}{1 - \exp(L)} \end{aligned} \quad (31)$$

Since $x = \frac{D}{C} > 0$, $y = \frac{D}{1 - \exp(L)} > 0$. $xe^x \geq y \iff x \geq W_0(y)$, we have:

$$C \leq \frac{D}{W_0\left(\frac{D}{1 - \exp(L)}\right)} \quad (32)$$

When $C = C_{\min}$, according to Lemma 3, it holds that $P_{\text{all}} \leq 2^{-\alpha}$. Since our objective is to identify the maximum value of C , it suffices to restrict the search to all candidates $C \geq C_{\min}$.

Table. III-E2 presents the optimal committee size factor C under $P_{\text{select}} = 2^{-100}$, across varying target committee sizes (N_{target}) and security parameters (α). Notably, except in extreme parameter regimes (e.g., large α combined with small N_{target}), the derived values of C consistently exceed 1. This implies that the consensus committee can be partitioned into at least two subcommittees while still satisfying the required security.

$n_{\text{target}} \backslash \alpha$	20	25	30	35	40	45	50	55
1000	4	3	3	2	2	2	1	1
1500	7	5	4	4	3	3	2	2
2000	9	7	6	5	4	4	3	3
2500	11	9	7	6	5	5	4	4
3000	13	10	9	7	6	6	5	5

TABLE II
THE OPTIMAL VALUES OF C FOR DIFFERENT N_{TARGET} AND α WHEN $P_{\text{SELECT}} = 2^{-100}$

Notably, both the Chernoff inequality [?] and Bernoulli inequality [?] are quite tight. Thus, the upper bound on P_{all} derived in Lemma 3 serves as a close approximation to the true P_{all} . Furthermore, P_{all} displays a roughly monotonic dependence on C . Consequently, in computationally constrained settings, it is reasonable to directly use the $C_{\min}$ given in Theorem 5 as the optimal value of C .



Fig. 2. α -bit security threshold v.s. subcommittee number (C) when $P_{\text{select}} = 2^{-100}$, $p = 0.33$ and $N_{\text{target}} = 2000$

Fig. 2 shows the optimal value of C , together with its theoretical upper and lower bounds, as a function of α . The optimal solution consistently remains within the feasible interval defined by the maximum and minimum possible values. Moreover, in agreement with the analysis presented

earlier, the optimal value of C closely tracks the lower bound, indicating that the minimum feasible solution provides a tight approximation to the optimum.

IV. IMPLEMENTATION AND EVALUATION

Next, we implement the proposed `FastOracle`, and evaluate its performance in terms of request processing time, request processing throughput, queue size of request pool, and the number of generated certificates.

A. Experimental Setup

Basic Settings. We conduct our `FastOracle` prototype in Golang and evaluated its performance on PoS blockchain and PoW blockchain, respectively. The prototype was deployed on a Windows system with an Intel CPU Core i7-12700H (2.30GHz) and 32GB of DDR5 RAM.

All experiments were conducted with a oracle network size of $N = 2000$ nodes. Following the committee size analysis in Section III-E, `FastOracle` organizes nodes into 4 parallel committees, each comprising 440 nodes. The maximum number of Byzantine nodes was fixed to $\lfloor \frac{N-1}{3} \rfloor$.

Each experiment consisted of 20,000 requests injected at a fixed rate of 100 requests/sec. External data were fetched via HTTPS from the Shenzhen Stock Exchange [?] under a constrained downlink bandwidth of 100 Mb/s. Requests were processed in batches of 15 as the Shenzhen Stock Exchange [?] imposes limits on parallel HTTPS requests per IP. The systems run continuously until all requests are confirmed by the blockchain. We added a delay ranging from 1 second to 2 second to each message to simulate the geographic distribution of nodes.

Chain Settings. We deployed an Ethereum-compatible blockchain using Ganache [?]. To emulate representative PoS and PoW environments, we configured average block intervals of 12 seconds (Ethereum-like PoS) and 600 seconds (Bitcoin-like PoW), respectively.

Baseline Solutions. For comprehensive benchmarking, we also implemented four state-of-the-art oracle schemes in Golang:

- **SenFeed [?]:** A dynamic decentralized oracle for IoT streaming data. It employs incremental truth discovery for real-time accurate inference and a novel BFT consensus protocol supporting dynamic node membership without system resets.
- **DeepThought [?]:** A human-in-the-loop oracle that certifies off-chain information via distributed voting and reputation mechanisms, achieving stronger resistance to voter corruption than prior reputation-based systems.
- **DECENTRUTH [?]:** A truth-discovery oracle addressing deceitful but authentic sources. It combines off-chain truth discovery with asynchronous BFT consensus to deliver highly accurate estimated truths.
- **DAON [?]:** A fully autonomous oracle network with a dedicated consensus protocol, non-interactive reputation and payment schemes, and continuous security monitoring, ensuring tamper-proof data feeds for smart contracts in complex Byzantine environments.

These baselines enable a fair and reproducible comparison with `FastOracle` under identical hardware, network, and workload conditions.

Metrics. We evaluate our `FastOracle` using the following four metrics: **i) Request processing time.** The time required to process a single request, measured from the moment when the request is submitted to the oracle network until the requested data is successfully confirmed on-chain. **ii) Request processing throughput.** The number of requests processed per second by oracle system. **iii) Queue size of request pool.** The number of pending requests in the oracle request pool. **iv) Queueing delay time.** The time each request spends waiting in the queue before being processed. **v) Cumulative number of generated signature.** Cumulative number of signature generated by oracle nodes.

B. Experiment Result

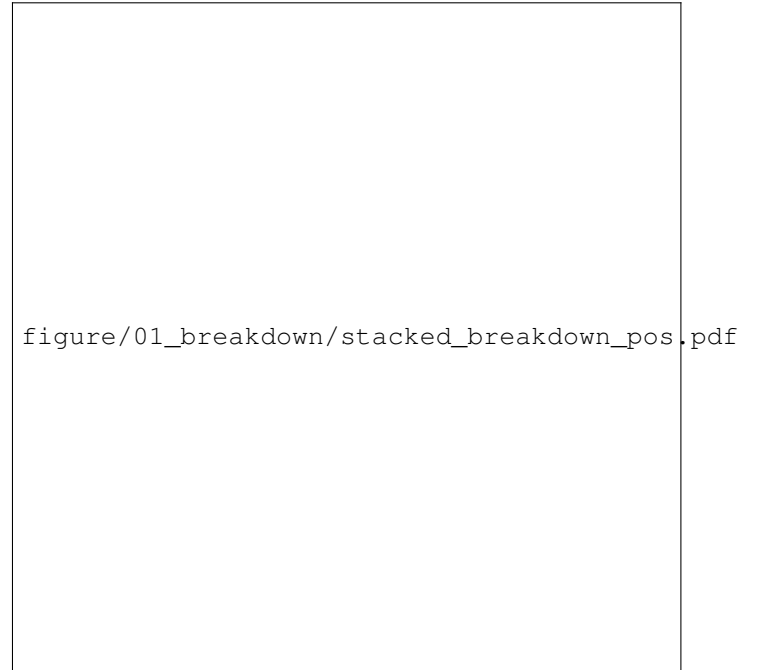


Fig. 3. POS latency

1) Latency: We first evaluate latency of oracle request processing by dividing it into three phases: (1) data retrieval, (2) off-chain consensus, and (3) on-chain confirmation. For each processed request, we measure the duration of these phases and compute the per-phase average latency across all requests for each oracle schemes. The results are shown in Fig. 4 (PoW chain) and Fig.3 (PoS chain).

As shown in figure, data retrieval time and on-chain confirmation time are comparable across all schemes. However, our scheme exhibits lower off-chain consensus latency than the other oracle schemes except DAON [?], since the security of our `FastOracle` relies on the underlying blockchain, eliminating the need for redundant consensus mechanisms required by other oracle schemes to ensure security.

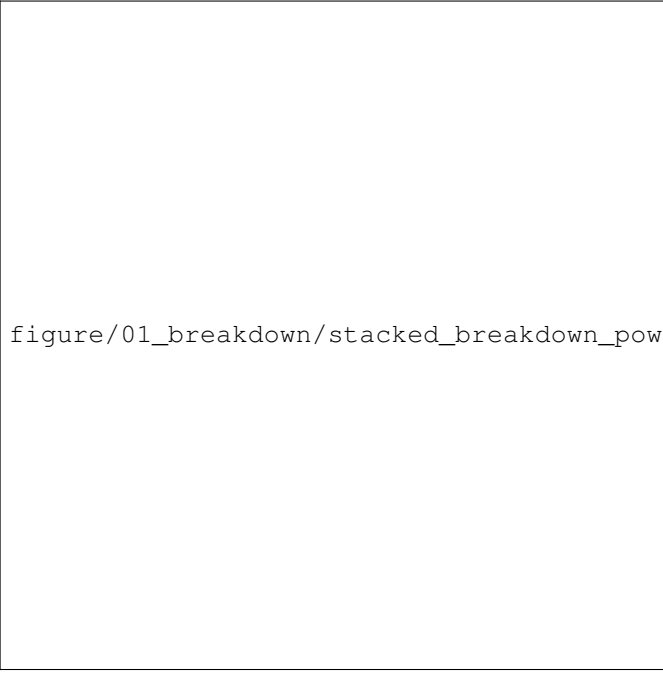


Fig. 4. POW latency

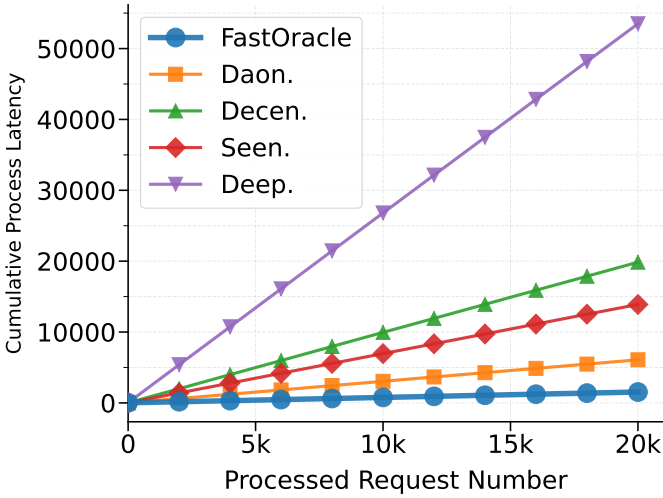


Fig. 5. POS

2) *Throughput vs. Latency*: We further measured the relationship between the number of processed requests and total latency. Specifically, we evaluate the total latency for each scheme when handling 1,000, 3,000, 6,000 and 10,000 requests on both PoW and PoS chains. The results are presented in Fig.5 for PoS chain and Fig.6 for PoW chain.

As shown in the figures, the total latency increases with increase of the number of request. Obviously, FastOracle consistently outperforms the baseline schemes across all evaluated request volumes. For instance, FastOracle processes 3000 requests in 232 seconds, while the fastest baseline (which is DAON [?]) requires 987 seconds. This trend holds across the entire range of request numbers. These results show

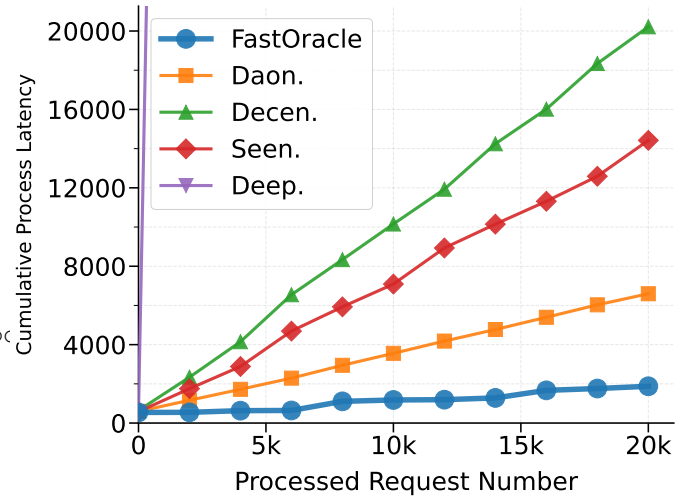


Fig. 6. POW

that FastOracle achieves significantly higher throughput compared to the baseline schemes on both PoS and PoW chains.

3) *Throughput*: We evaluate the throughput of the compared oracle schemes by tracking the cumulative count of requests processed every 20 seconds throughout the experiment, and plotted the results in Fig.7a and Fig.8a for the PoS and PoW chain, respectively.

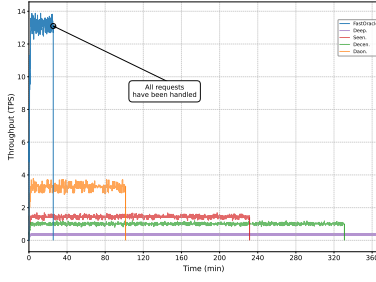
We easily observe that FastOracle reaches zero throughput significantly earlier than other baseline schemes, indicating that it requires less time to handle all requests. Moreover, FastOracle exhibits markedly higher throughput compared to the other schemes. In particular, FastOracle achieves approximately $3\times$ faster than the best baseline (DAON [?]) on both POW and POS chain.

4) *Queue Size*: We measure the queue size of the request pool of various oracle schemes to assess the performance of processing requests. Fig. 7b and Fig. 8b represents how the queue size of oracle evolves as the system's runtime progresses on PoS and PoW chains, respectively. We find that FastOracle's queue size decreases faster than the other baseline schemes, demonstrating that FastOracle achieves higher request processing throughput. Notably, FastOracle's queue size converges to zero the fastest compared to all baselines, indicating that it achieves the lowest latency for processing the entire requests among all schemes on both PoS and PoW chains.

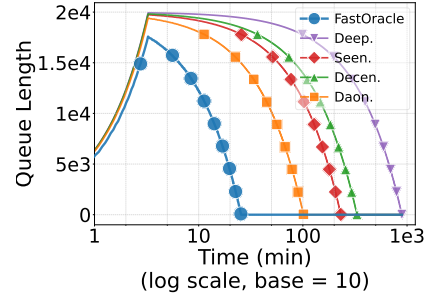
5) *Certificate*: As DeepThought [?] relies on on-chain consensus and does not produce off-chain certificates, we exclude it from this evaluation.

Fig.7c and Fig.8c presents the cumulative number of generated signature across different schemes in POS and POW chain, respectively. The x-axis denotes time in seconds, while the y-axis represents the cumulative number of generated signature.

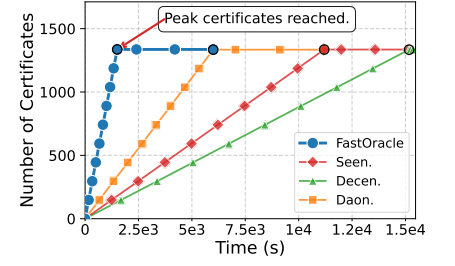
V. CONCLUSION



(a) Throughput

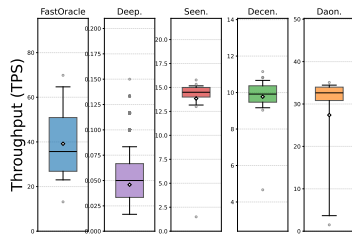


(b) Queue Size

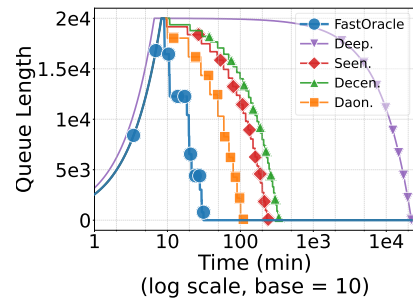


(c) Number of Signature

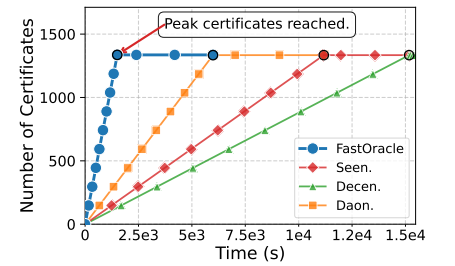
Fig. 7. POS Performance.



(a) Throughput



(b) Queue Size



(c) Number of Signature

Fig. 8. POW Performance.